



Testing Exam

The Road to A+

Lecture 1



CMM levels

1. Level One : Initial – Work is performed informally.

A software development organization at this level is characterized by AD HOC عشوائي activities (organization is not planned in advance.).

2. Level Two : Repeatable – Work is planned and tracked.

This level of software development organization has a basic and consistent project management processes to TRACK COST, SCHEDULE, AND FUNCTIONALITY. The process is in place to repeat the earlier successes on projects with similar applications.

3. Level Three : Defined – Work is well defined.

At this level the software process for both management and engineering activities are DEFINED AND DOCUMENTED.

4. Level Four : **Managed** – Work is quantitatively controlled.

- **Software Quality management** – Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance.
- **Quantitative Process Management** – At this maturity level, The performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.

5. Level Five : **Optimizing** – Work is Based Upon Continuous Improvement.

The key characteristic of this level is focusing on CONTINUOUSLY IMPROVING PROCESS performance.

Key features are:

- Process change management
- Technology change management
- Defect prevention

Error/Defect

- Statistical Software Process Improvement (SSPI)
- **Error**
 - Some flaw ثغرة in a s/w engineering work product that is uncovered before the s/w is delivered to the **end-user**
- **Defect**
 - A flaw that is uncovered after delivery to the **end-user**

1-Size-Oriented Metrics

- Lines of Code (LOC) can be chosen as the normalization value
- Example of simple size-oriented metrics
 - Errors per KLOC (thousand lines of code)
 - Defects per KLOC
 - \$ per KLOC
 - Pages of documentation per KLOC

1. Buffer Overflow

- Buffer Overflow occurs when data is written into a buffer (like array) past its end.
- It may arise due to faulty calculations of the write position. Or continuous writing into a buffer without checking the length.
- Problems:
 - Runtime Exception
 - Or runtime attack (security problem)

2.SQL Injection

- **SQL Injection** is a technique for injecting SQL commands into user input such that these commands are directly executed by the database.
- This allows the attacker to perform malicious acts such deleting tables, dropping databases, stealing data and much more.

3.OS Command Injection

- OS Command Injection arises when user-specified input is directly handed over to the operating system for execution by the application without proper vetting.
- Such an operation might be used by an application to use an existing command on the OS.
- When the application passes user input without properly validating it, it paves the way for an attacker to use clever constructs to execute malicious commands.
- These commands can be, for example, to **delete** files, **copy** data, **alter permissions** on files, and more.

12. Javascript == and === signs

- In this example:

`0 == '0'`

- the == operator considers the string to be integer, because it will make the two values the same type prior to comparing.
- This can lead to all sorts of problems that are difficult to track down.
- <https://www.dummies.com/web-design-development/javascript/10-common-javascript-bugs-and-how-to-avoid-them/>

13. Confusing object comparison (== instead of .equals)

- The == operator compares two objects physically (their memory addresses) whereas the equals() method compares two objects semantically (their information).
- And in most of the cases, we should compare two objects meaningfully using the equals() method.
- Novice programmers often make a mistake by comparing two String objects like this:

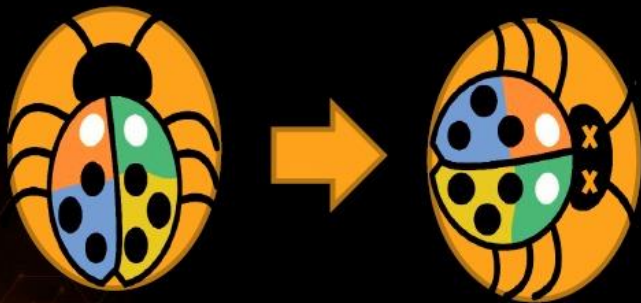
Lecture 2



What is Debugging?



- The process of locating and fixing bugs (errors) in program code
- Debugging tools (called **debuggers**)

A screenshot of a C# code editor window titled "Program.cs". The code shows a static void Main method with a for loop that iterates from 0 to 99, printing each value to the console. A green vertical bar on the left indicates the current execution point. The console output shows "1" and "2" with a "≤ 1ms elapsed" indicator. The bottom of the window shows the "Autos" and "Call Stack" panels.

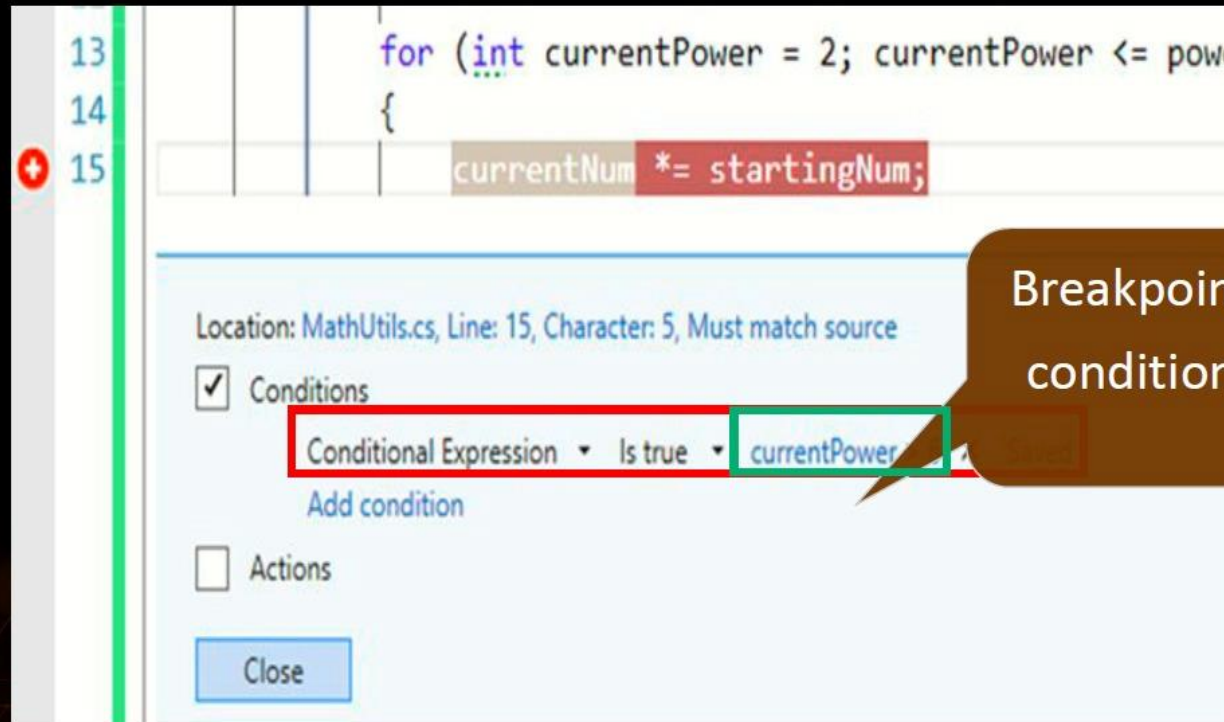
```
Program.cs * X
CSharpConsoleApp CSharpConsoleApp.Program Main(string[] args)
6 static void Main(string[] args)
7 {
8     for (int i = 0; i < 100; i++)
9     {
10         Console.WriteLine(i); ≤ 1ms elapsed
11     }
12 }
13
```

Name	Value	Type
Autos		
Call Stack		

Conditional Breakpoints



- Stop execution **only** when the set condition is true



Minification

Minification, also known as minimization, is the process of removing all unnecessary characters from JavaScript source code without altering its functionality.

This includes the:

1. removal of whitespace, comments, and semicolons,
2. along with the use of shorter variable names and functions.

Minification of JavaScript code results in compact file size.

Example: the following code contains 8 lines of code Before minification

```
1 // This function takes in name as a parameter
2 // and logs a string which greets that name
3 // using the information passed
4 function sayHi (name) {
5     console.log ("Hi" + name + ", nice to meet you." )
6 }
7
8 sayHi("Sam");
```

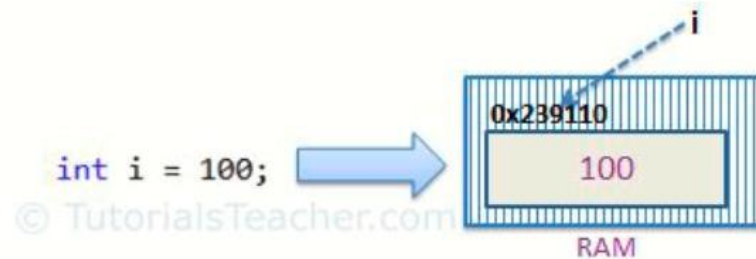
After minification: A single line of code

```
1 function sayHi(o){console.log ("Hi"+o+", nice to meet you.") }sayHi("Sam");
```

<https://www.cloudflare.com/learning/performance/why-minify-javascript-code/>

Value type, and Reference type,

- ▣ **Value Type:** A data type is a value type if it holds a data value within its own memory space. It means variables of these data types directly contain their value
- ▣ The following data types are all of value type: (c#)
 - bool
 - byte
 - char
 - decimal
 - double
 - enum
 - float
 - int
 - long
 - sbyte
 - short
 - struct
 - uint
 - ulong
 - ushort

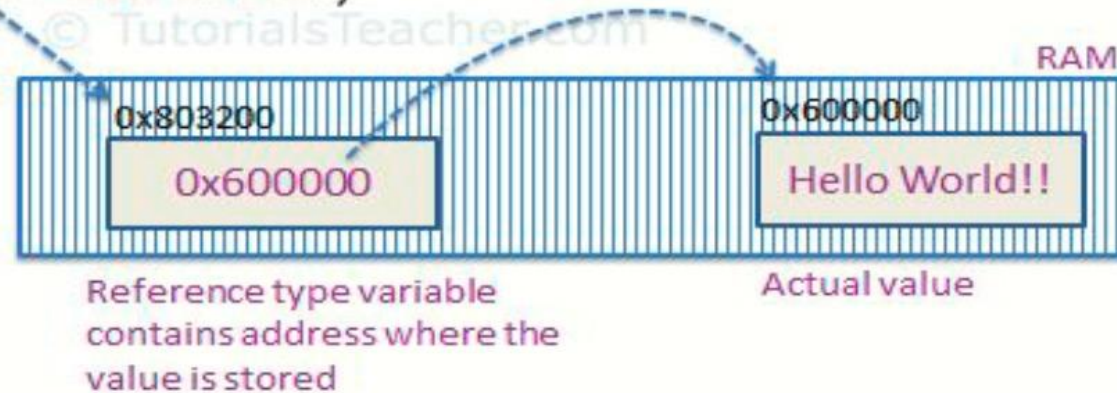


Value type, and Reference type,

▣ Reference Type

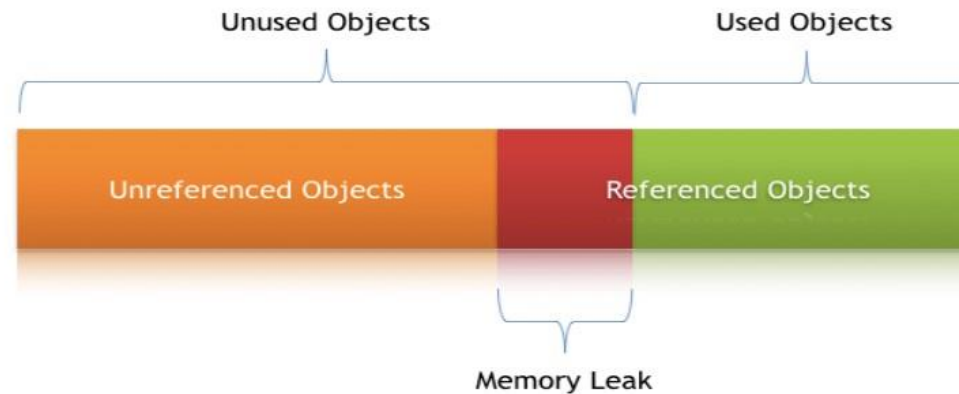
- Unlike value types, a reference type doesn't store its value directly. Instead, it stores the address where the value is being stored. In other words, a reference type contains a pointer to another memory location that holds the data.
- Each Referenced Object contains 2 parts:
 - ▣ Object: the real data holder in heap

```
string s = "Hello World!!";
```



Memory Leak

- A Memory Leak is a situation when there are **objects present in the heap** that are **no longer used**, but the garbage collector is unable to remove them from memory and, thus they are unnecessarily maintained.
- A memory leak is bad because it blocks memory resources and degrades system performance over time.
- And if not dealt with, the application will eventually exhaust its resources, finally terminating with a fatal `java.lang.OutOfMemoryError`.



<https://www.baeldung.com/java-memory-leaks>

Profilers

- Wikipedia:
 - In software engineering, profiling ("program profiling", "software profiling") is a form of **dynamic program analysis** that **measures**, for example:
 - the **space** (memory)
 - **time complexity** of a program,
 - **the usage of particular instructions**,
 - or the **frequency** and **duration** of function calls.
 - Most commonly, profiling information serves to aid program optimization.
- Profilers are tools to perform code Profiling
 - Examples:
 - DotTrace (.net)
 - Netbeans Profiler, Visual VM (java)
 - Android studio Profiler
 - firefox/chrome browser profilers
 - Unity Profiler
 -

Lecture 3&4



Code Visibility Levels During Testing

• There are 3 levels of Code Visibility:

1. **Black box testing** (also called functional testing):
is testing that ignores the internal code of a system or component and focuses solely on the **outputs** generated in response to **selected inputs** and **execution conditions**.
2. **White box testing** (also called structural testing and glass box testing):
is testing that takes into account the **full internal code** of a system or component.
3. **Gray Box testing**:
partial interaction of internal code of system/component.

Black Box

No insight into the code and underlying architecture

White Box

Full understanding of the code and the software architecture

Gray Box

Some insight into the code and underlying architecture

Types of Test Techniques

1-Static Analysis Techniques –

- Based solely on the (manual or automated) examination of project documentation of software models and code, and of other related information about requirements and design
- Generally yields valid results, but may be weak in precision
- فحص ملفات التوثيق والكود بدون تشغيل البرنامج

2-Dynamic Test Techniques –

- Test the software Execution in order to find possible failures
- Behavioral and performance properties of the program are also observed
- Yields more precise results, but only holding for the examined execution
أدق ولكن في الحالات المختبرة فقط

detailed comparison between Unit and integration testing

Unit testing	Integration testing
Unit testing focuses on the individual modules of the application.	Integration testing focuses on the combined modules of the application.
It is usually the first level of testing but can be performed at any time.	It is performed after Unit testing and before System testing.
It can be performed by developers, testers, and QA engineers .	Only performed by testers .
It is a white-box testing technique.	It is a black-box testing technique.
It can be carried out without the completion of all the parts of the software.	Only be carried out after the completion of all the parts of the software.
It is easy to maintain, run and debug.	It's comparatively high maintenance and slower to run.
The issues are easy to find and can be instantly fixed.	The cost of fixing issues is higher and takes longer to resolve .
It focuses on module specification.	It focuses on interface specification.

Verification and Validation

- Software testing is part of a broader group of activities called verification and validation that are involved in software quality assurance
- **Verification (Are the algorithms coded correctly?)**
 - The set of activities that ensure that software correctly implements a specific function or algorithm
- **Validation (Does it meet user requirements?)**
 - The set of activities that ensure that the software that has been built is traceable to customer requirements

NUnit – Attributes

- [TestFixture] - This attribute marks a **class** that contains tests, and, optionally, setup or teardown methods
- [Test] - The Test attribute marks a **specific method** inside a class that has already been marked as a TestFixture, as a test method

Lecture 5



Web Engineering Project Metrics (2)

- Let,
 - N_{sp} = number of static Web pages
 - N_{dp} = number of dynamic Web pages
- Then,
 - **Customization index**, $C = N_{dp} / (N_{dp} + N_{sp})$
- The value of C ranges from 0 to 1

Lecture 6



SQL injection prevention techniques

Primary Defenses:

- Option 1: Use of Prepared Statements (with **Parameterized Queries**)
- Option 2: Use of Properly Constructed **Stored Procedures**
- Option 3: Allow-list **Input Validation**
- Option 4: **Escaping All** User Supplied Input

Additional Defenses:

- Also: Enforcing Least Privilege
- Also: Performing Allow-list Input Validation as a Secondary Defense
- Always perform 'white list' input validation on all user supplied input

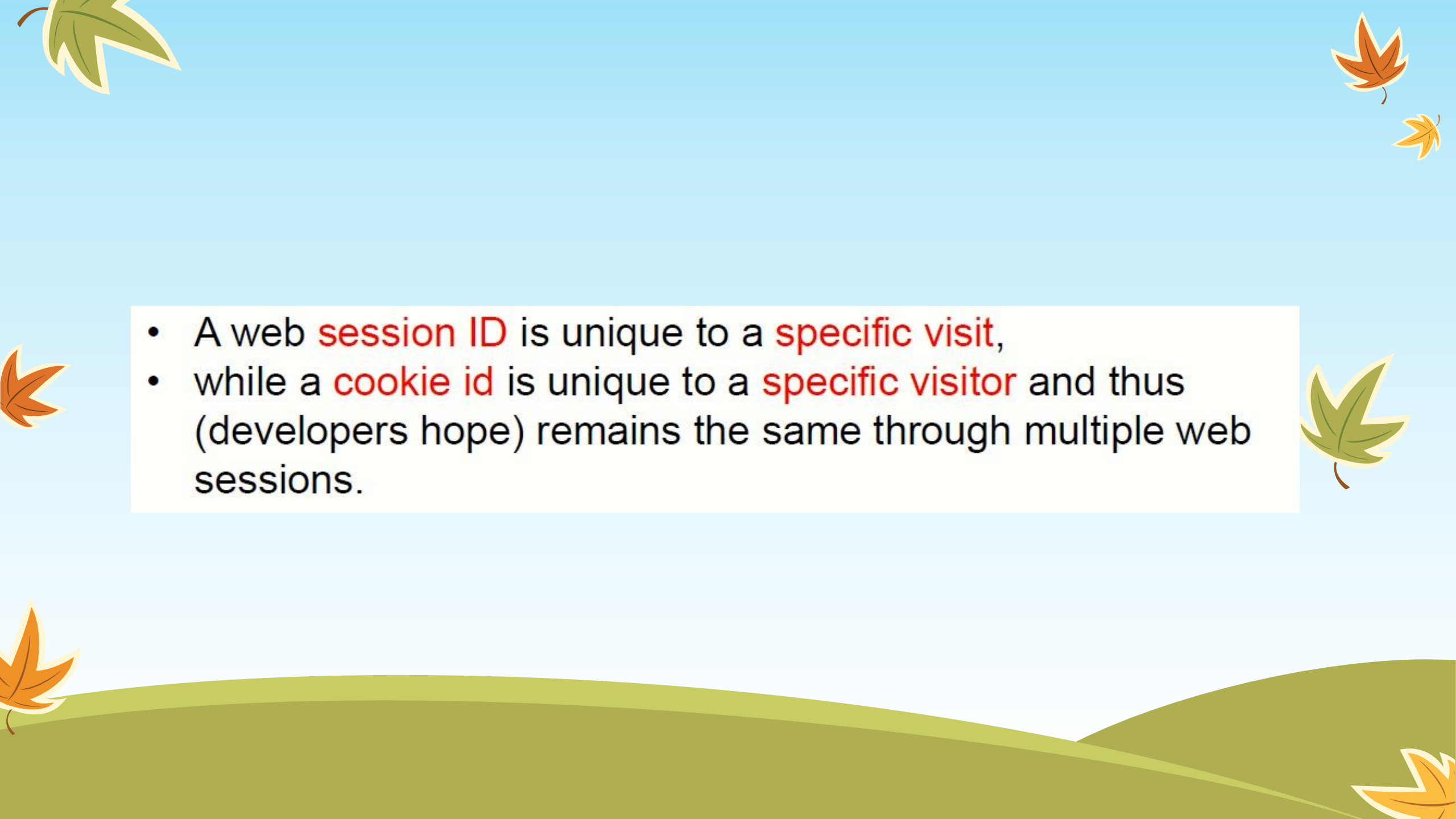


A cookie is a small piece of data from a web site that is stored on a visitor's browser to help the website track the visitor's activity on the web site. Sessions and cookies are sometimes conflated, creating confusion. More specifically, session IDs and cookie IDs are confused.



While they are closely related, they are not the same thing. A cookie identifies, often anonymously, a specific visitor or a specific computer. Cookies can be used for authentication, storing site preferences, saving shopping carts, and server session identification



- 
- The slide features a light blue background with stylized autumn leaves in green and orange scattered around the edges. At the bottom, there are rolling green hills. A central white box contains a bulleted list.
- A web **session ID** is unique to a **specific visit**,
 - while a **cookie id** is unique to a **specific visitor** and thus (developers hope) remains the same through multiple web sessions.

session management types

There are two main types of web session management: client-side and server-side.

1. **Client-side session management** stores the session data on the client's browser. This is typically done using cookies, which are small pieces of data that are stored on the client's computer. When the client makes a request to the server, the cookie is sent with the request. The server can then use the cookie to identify the client and retrieve the associated session data.
2. **Server-side session management** stores the session data on the server. When a client connects to the server, the server generates a unique session identifier and sends it to the client. The client stores the session identifier and sends it back to the server with each request. The server then uses the session identifier to retrieve the associated session data.

1-Client-side session management methods

- **Cookies:** Cookies are the most common type of client-side session management. They are simple to implement and use, and they are supported by all major browsers. However, cookies can be disabled by users, and they can be stolen by attackers.
- **Hidden form fields:** Hidden form fields are another way to store session data on the client side. When a client submits a form, the hidden form fields are sent to the server with the request. The server can then use the hidden form fields to identify the client and retrieve the associated session data. However, hidden form fields can be easily tampered with by attackers.
- **Browser storage:** Browser storage mechanisms such as local storage and session storage can also be used to store session data on the client side. Browser storage is more secure than cookies because it cannot be accessed by other websites. However, browser storage is not supported by all browsers.

2- Server-side session management methods

- **URL rewriting:** URL rewriting is a technique that can be used to store the session identifier in the URL. When a client makes a request, the server rewrites the URL to include the session identifier. The client then sends the rewritten URL to the server with each request. However, URL rewriting can be cumbersome to implement, and it can be vulnerable to attacks.
- **HTTP headers:** HTTP headers can also be used to store the session identifier. When a client makes a request, the server sends the session identifier in an HTTP header. The client then sends the HTTP header to the server with each request. However, HTTP headers can be tampered with by attackers.
- **In-memory storage:** In-memory storage is a simple way to store session data on the server. The server stores the session data in memory and retrieves it when needed. However, in-memory storage is not scalable, and it can be lost if the server crashes.
- **Database storage:** Database storage is a more scalable and reliable way to store session data on the server. The server stores the session data in a database and retrieves it when needed. However, database storage can be more complex to implement than other server-side session management methods.

A3 – Cross-Site Scripting (XSS)

- Attacker sends **text-based attack scripts** that exploit the interpreter in the browser. Almost any source of data can be an attack vector, including internal sources such as data from the database.
- Cross-site scripting attacks are those in which attackers inject malicious code **كود ضار**, usually client-side scripts, into web applications.
- Because of the number of possible injection locations and techniques, many applications are vulnerable **معرضة لخطر** to this attack method.

Cross Site Scripting (XSS)- types

1. **NonPersistent**

it requires a **user to visit the specially crafted link** by the attacker. When the user visit the link, the crafted code will get executed by the user's browser.

2. **Persistent**

it occur in either **community content driven websites** or Web **mail** sites and do not require specially crafted links for execution.

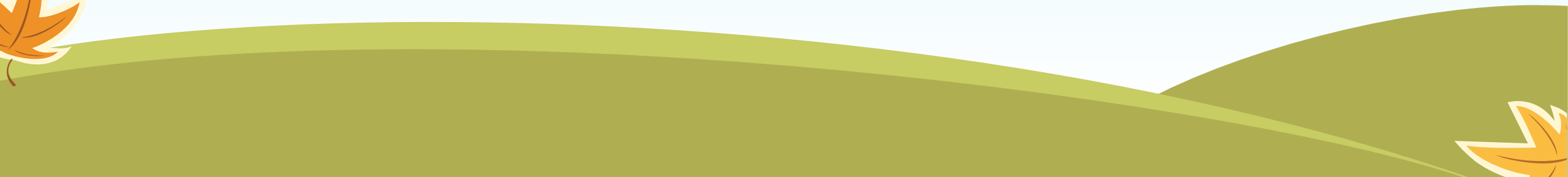
persistent XSS much more dangerous than non-persistent because the user has no means of defending himself. Once a hacker has his exploit code in place, he'll again advertise the URL to the infected

- Two pages have the **same origin** if the following are the same in both pages:
 - **Protocol**
 - **port** (if one is specified),
 - and **host**
- The following table gives examples of origin comparisons to the URL **`http://store.company.com/dir/page.html`**:

URL	Outcome	Reason
<code>http://store.company.com/dir2/other.html</code>	Success	
<code>http://store.company.com/dir/inner/another.html</code>	Success	
<code>https://store.company.com/secure.html</code>	Failure	Different protocol
<code>http://store.company.com:81/dir/etc.html</code>	Failure	Different port
<code>http://news.company.com/dir/other.html</code>	Failure	Different host



What is frame busting?

- Frame busting are techniques for preventing framing by the framed site.
- 
- 
- 

Lecture 7



Performance Testing Metrics: Parameters Monitored - 1

- **Processor Usage** – an amount of time processor spends executing non-idle threads.
- **Memory use** – amount of **physical** memory available to processes on a computer.
- **Disk time** – amount of time disk is busy executing a read or write request.
- **Bandwidth** – shows the bits per second used by a network interface.
- **Private bytes** – number of bytes a process has allocated that **can't** be shared amongst other processes. These are used to measure memory leaks and usage.
- **Committed memory** – amount of **virtual** memory used.
- **Network bytes total per second** – rate which bytes are sent and received on the interface including framing characters.
- **Response time** – time from when a user enters a request until the first character of the response is received.

Response Time and Latency

- **Response time** = time for an operation **to complete**, **including** the time spent **waiting** and being **serviced**
- **Latency** = time an operation spent **waiting** in a **queue**
- **Service time** = **Actual time** to perform job continuously.
- **Response time = Latency + Service time**
- Easily quantifying degradation and improvement
- Interpretation depends on the purpose of the operation

IOPS

- Input / output operations completed per second
- Throughput-oriented metric
- The meaning depends on the target evaluated
- Examples:
 - Network devices (TCP) – packets received / sent per second
 - Block devices – reads / writes per second

Utilization

There are 2 definitions:

1. **Time-based definition:** How much **time** a resource is busy during a period of time.
 - CPU Utilization
 - Disk Utilization
 - 100% Utilization doesn't have to lead to the saturation of the resource. We need to examine other metrics of the resource that indicates its saturation
2. **Capacity-based definition:** The extent to which the capacity مقدار استغلال القدرات المتاحة of a resource was used during a time period.

Saturation

- The extent to which a resource has **queued** work because it **cannot accept** more work التعطل بسبب الوصول للذروة
- Capacity-based Utilization $\geq 100\%$
- Increases latency and response time
- **Bottleneck** = the **resource** that **limits** performance

Note about capacity utilization

- it assumes that every resource (component) has some maximum capacity it can deliver.
- However, it's very **difficult to determine** the maximum capacity for some resources,
- for instance hard disks. To determine the maximum capacity of hard disks, the other parts of computer systems that disks use must be evaluated.

More Types of Performance Testing

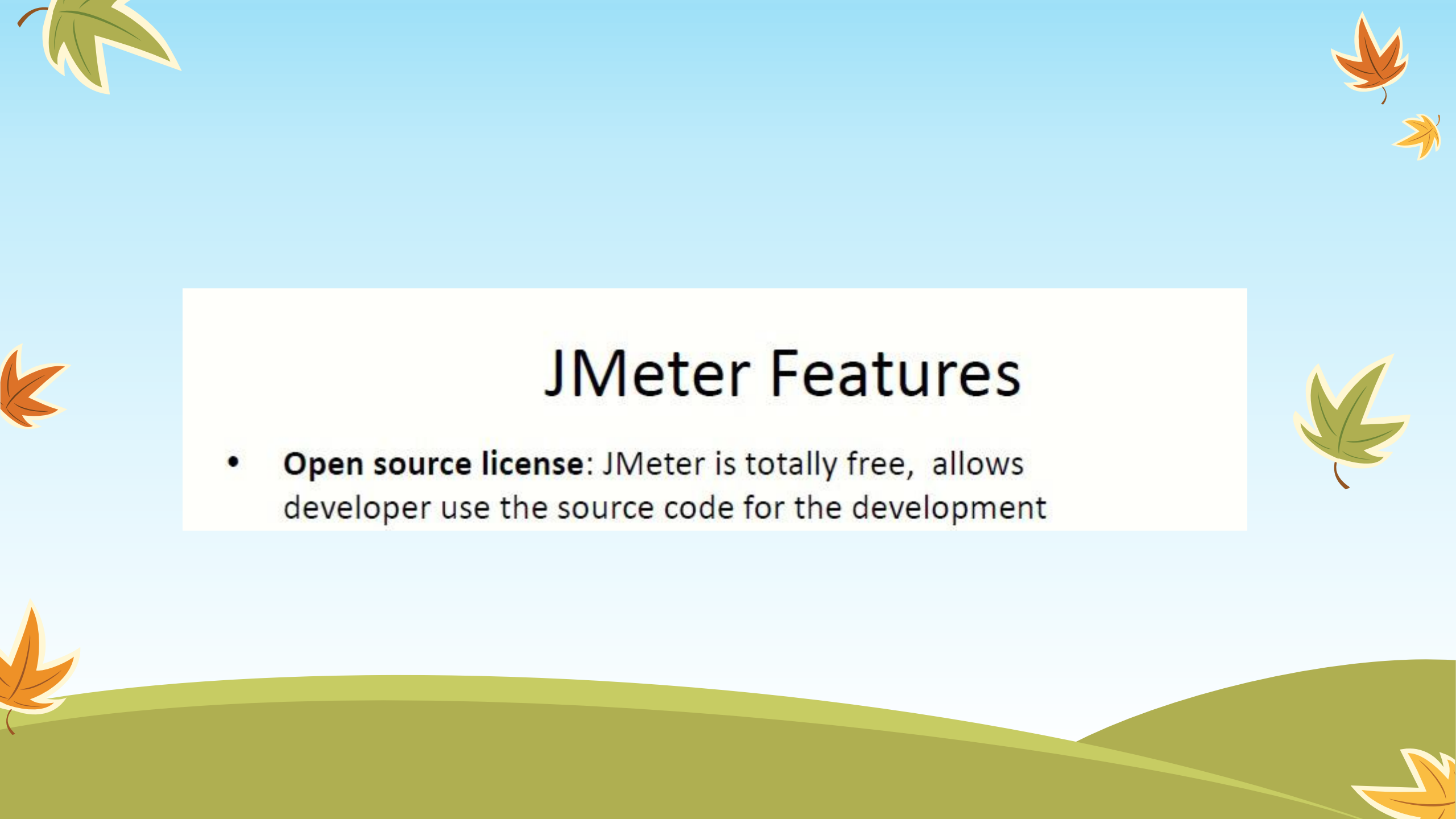
Public

- **Spike testing** – tests the software's reaction to **sudden large spikes** ارتفاع مفاجئ في الأحمال in the load generated by users.
- **Volume testing** – Under Volume Testing **large number of Data** is populated in a database, and the overall software system's behavior is monitored. The objective is to check software application's performance under varying database volumes.

<https://www.guru99.com/performance-testing.html>

Load & Stress Testing

- JMeter Performance Testing includes:
 1. **Load** Testing: Modeling the **expected usage** by simulating multiple user access the Web services concurrently.
 2. **Stress** Testing: Every web server has a maximum load capacity. When the load **goes beyond** تجاوز الحد الأقصى the limit, the web server starts responding slowly and produce errors. The purpose of the Stress Testing is to **find the maximum load** the web server can handle.

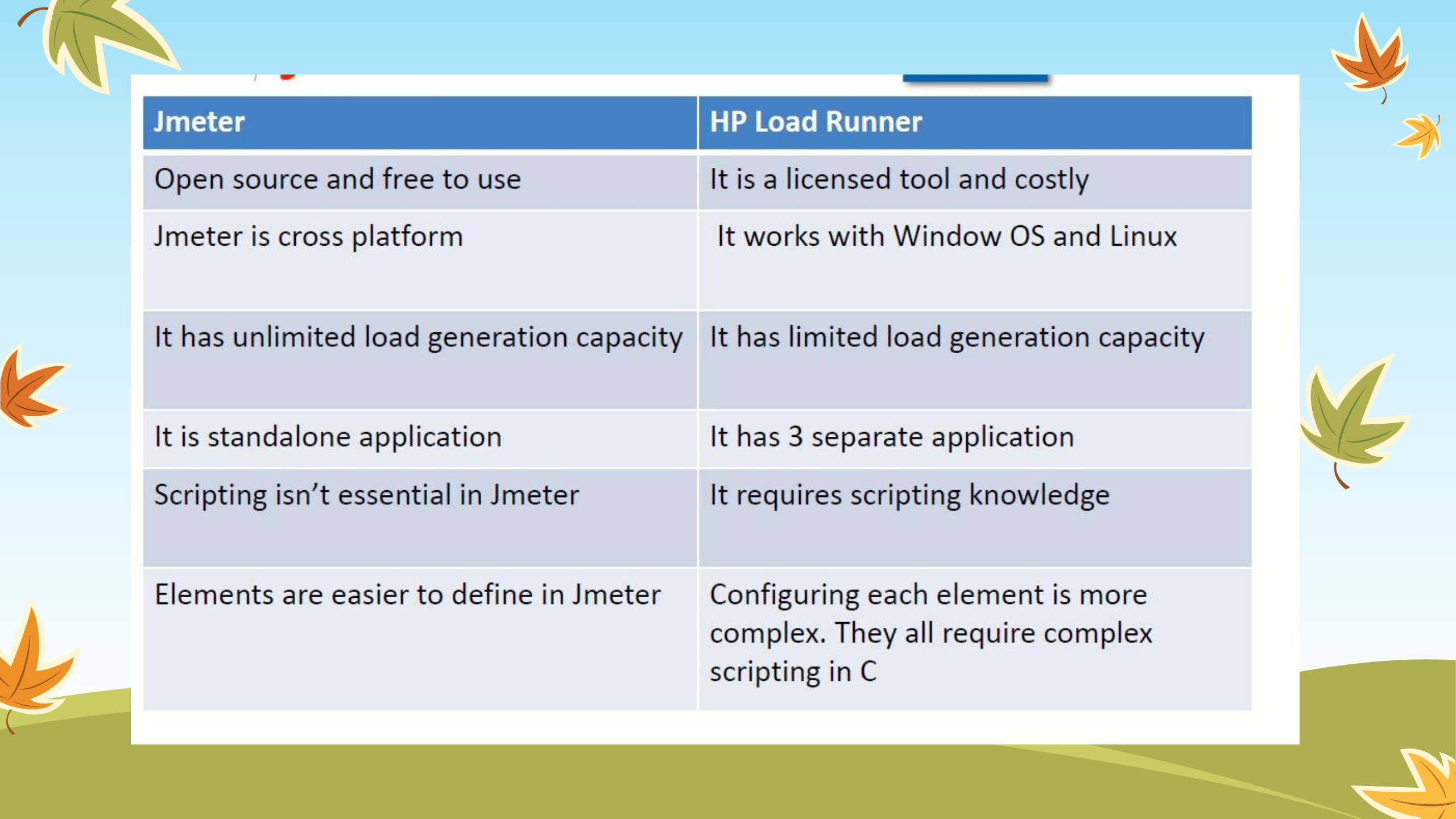
The slide features a light blue background with stylized autumn leaves in green, orange, and yellow scattered around the edges. At the bottom, there are rolling green hills. The title 'JMeter Features' is centered in a large, bold, black font.

JMeter Features

- **Open source license:** JMeter is totally free, allows developer use the source code for the development

- **Support multi-protocol:** JMeter does not only support web application testing but also evaluate database server performance. All basic protocols such as HTTP, JDBC, LDAP, SOAP, JMS, and FTP are supported by JMeter





Jmeter	HP Load Runner
Open source and free to use	It is a licensed tool and costly
Jmeter is cross platform	It works with Window OS and Linux
It has unlimited load generation capacity	It has limited load generation capacity
It is standalone application	It has 3 separate application
Scripting isn't essential in Jmeter	It requires scripting knowledge
Elements are easier to define in Jmeter	Configuring each element is more complex. They all require complex scripting in C

2. Create test plan

A useful test plan is created with minimum 3 components –

- **Thread Group** –
 - contains the simulation of multiple concurrent users.
 - **A single thread represent a single user.**
 - We can create any number of threads to put the desired load on the application.
 - It also help us in scheduling the delay between two threads, and any repetition of request batches.
- **HTTP Request** –
 - consist the HTTP request configuration which thread group will be invoking.
 - It is **the application URL** which you want to load test.
- **Listener** –
 - helps in viewing the **result** of the **whole** testing process.
 - There are multiple listener available in jmeter to verify the testing results.

<https://howtodoinjava.com/library/jmeter-beginners-tutorial/>

Lecture 8



NATIVE APPS-2

PROS
Easy low-level hardware access services.
Easy access to high level services important to personal mobile experience.
Full use of all functionalities that modern mobile devices have to offer.
High usability.

CONS
Code Reusability : Low
Development & maintenance: Time-consuming & .expensive
Designers are required to be familiar with different UI components of each OS.
.Upgrade flexibility: Low

1-CROSS-COMPILATION

- Separates **build environment** from **target environment**.
- **Platform-independent API** using a mainstream programming language like JavaScript, Ruby or Java.
- The **cross-compiler** then **transforms** the **code** into **platform-specific native** apps.
- The software artifact generated can be deployed and executed natively on the device.

ADVANTAGES:

- Improved **performance** and User Experience.
- **Full access** to functionalities of underlying mobile OS and device specific capabilities.

DISADVANTAGES:

- Highly **complex as cross-compilers** are **difficult** to program.
- Need to be keep **API consistent with different mobile platforms** and operating systems available.

3-MOBILE WEB APPS

- Use standard web technologies such as **HTML 5, CSS 3 & JavaScript**.
- Features of HTML 5 :
 - **Advanced UI components**,
 - access to **rich media** types,
 - **geolocation** services
 - **offline** availability.
- Increasing popularity of HTML 5 in rendering engines such as WebKit.
- Runs on a **standalone mobile web browser**.
- Installed **shortcut**, launched like a **native app**.
- UI logic resides locally; makes the app responsive and accessible **offline**.

ADVANTAGES:

- **Multiplatform** support.
- **Low** development cost.
- Leverage **existing knowledge of web technologies**.

DISADVANTAGES:

- **Limited access** to OS API's.

Real Device and Emulators Comparison

Criteria	Real testing device	Virtual testing device
Cost	Buying real devices at scale is cost prohibitive عائق مادي	Minimal cost incurred as oftentimes one can install them for free
Reliability	Real devices exhibit accurate results and allow for testing in the same condition as a user	Virtual testing devices only mimic تحاكي the device and can't replicate real user conditions like hardware and software configurations
Processing Speed	Software testing on real devices is much faster	Software testing on virtual devices is slower due to Binary translation
Suitable for Debugging	Debugging with real testing devices could be tricky , especially while capturing defects	Virtual devices make step by step debugging easy with the features, where you can capture the defects
Cross-Platform Testing	Cross-Platform Testing can be conducted normally using real devices	Cross-Platform Testing can be seamlessly conducted using virtual devices (works in a less degree than real devices)

Types of Mobile Ads

1-Mobile Video Advertising

Video advertising is a fast-growing industry because of its high conversion rates when promoting a product or service

2-Banner Advertising

- A [banner advertisement](#) is a thin horizontal graphic that **stretches** across a screen to promote a brand or product.

3-Pop-ups

- Pop-ups are windows that appear on your screen and overlay the host website's page. These are great for improving conversion rates and building email lists.

4-Native Advertisements

- Native advertisements are built to blend into the environment or app that they are displayed on. A significant advantage of native advertising is that ads can't be exited or blocked by the user.

5-Interstitial Advertisements

- [Interstitial advertisements](#) are used at transition points within apps and take up the **full screen** of your mobile device. Some examples of transition points occur while a page is loading, in between game levels, or when opening an app.

6-Location Advertising

- Location advertising (also known as [geotargeting](#)) is the process of sending targeted messaging to individuals that enter a specific location. Geotargeting is a great method for gathering potential customer data by seeing which stores they frequent and tapping into their interests.

What Is a Call to Action (CTA)?

- A call to action (CTA) is a marketing term that refers to the next step a marketer wants its audience or reader to take.
الخطوة التالية بعد مشاهدة المحتوى
- For example, it can instruct the reader to click the buy button to complete a sale, or it can simply move the audience further along towards becoming a consumer of that company's goods or services.

Types of Gaming Business models :

1. **Game purchase (Premium games):** where the digital gaming company sells it at a given price.
2. **In-app purchases (Freemium games):** similar to the free-to-play model, the free game prompts gamers to buy either digital goods or premium versions/upgrades of the game.
3. **Ad-supported:** another model is the ad-supported, where gamers get it for free, and advertising is rendered within the game, which will monetize on both an impression and click basis.
4. **Hybrid model games:** A mix between In-app purchases and Ad-display which lets you pay to get rid of the advertisements.
5. **non-game focused monetization :** like videos about game playing

Lecture 9

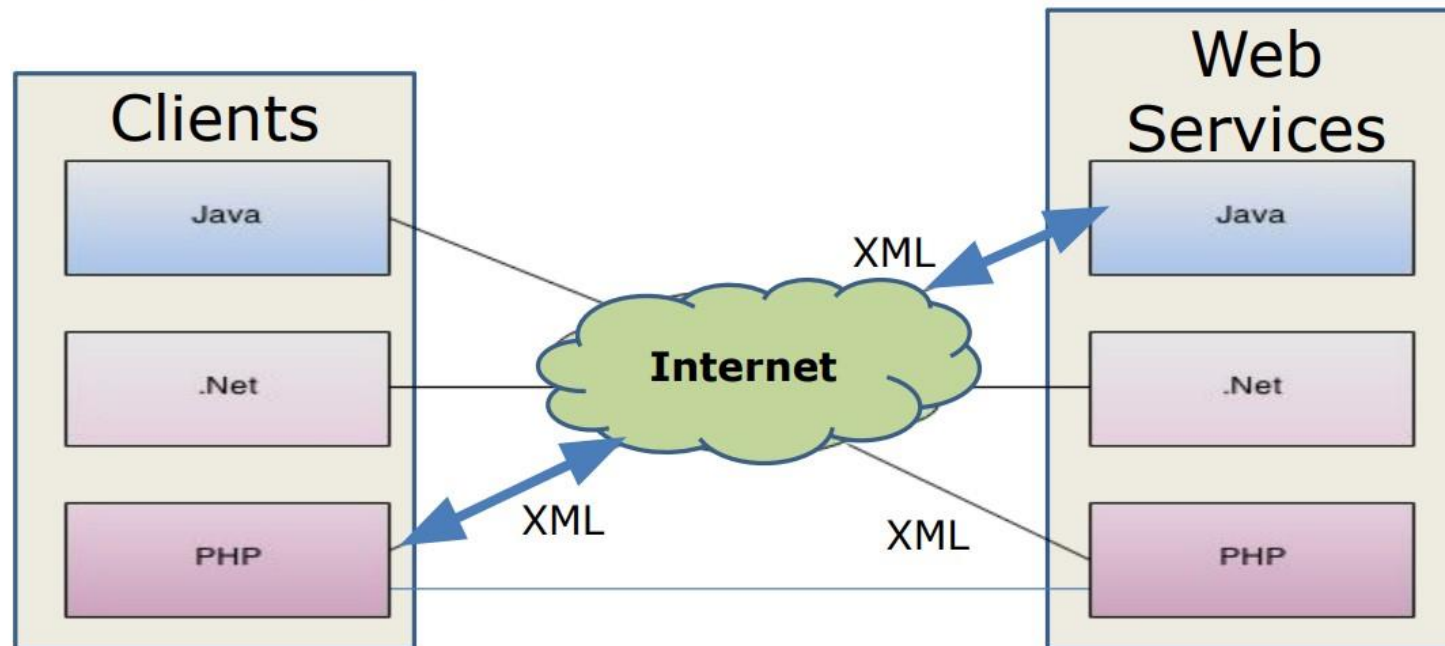


Web Service

- An application that requires interaction with another application
 - **Service provider**: the application that provides the service (server)
 - **Service consumer**: the application that consumes the service (client)
- Any program can be published as a web service if it does something!

Web Service

- A web service is any service that is available over the Internet or private (intranet) networks
- Uses a standardized XML messaging system
- A web service is
 - not tied to any one operating system or programming language
 - self-describing via a common XML grammar
 - discoverable via a simple find mechanism



Soap Vs Rest

REST API	SOAP
has no has no official standard at all because it is an architectural style.	has an official standard because it is a protocol.
uses multiple standards like HTTP, JSON, URL, and XML	SOAP APIs is largely based on HTTP and XML.
it takes fewer resources and bandwidth	uses XML for the creation of Payload (sent data) and results in the large sized file.
Faster	Slower

SOAP Web Services Protocols

- Web Services Definition Language (**WSDL**):
 - Allows a web service to create a standard description of its service
- Universal Description Discovery and Integration (**UDDI**):
 - Allows services to publish its description, and allows users to find it
- Simple Object Access Protocol (**SOAP**):
 - Allows a web service and service user/consumer to execute a service transaction

- **RESTful**

- REST stands for **RE**presentational **S**tate **T**ransfer. It is an architectural style not a protocol

- **Pros**

- **Fast:**
 - RESTful Web Services are fast because there is no strict specification like SOAP. It consumes **less** bandwidth and resource
 - **Language and Platform independent:**
 - RESTful web services can be written in any programming language and executed in any platform
 - **Can use SOAP:**
 - RESTful web services can use SOAP web services as the implementation
 - **Permits different data format:**
 - RESTful web service permits different data format such as **Plain Text**, **HTML**, **XML** and **JSON**
- 
- 
- 
- 

Understanding REST

- Resources
- **HTTP verbs**
- Status codes
- Media types

GET

'Read' a resource

POST

'Create' a resource

PUT

'Update or Create'

DELETE

'Delete' a resource

HEAD

'Read' resource headers

Understanding REST

- Resources
- HTTP verbs
- **Status codes**
- Media types

1xx

Informational

2xx

Success

3xx

Redirection

4xx

Client Error

5xx

Server Error

What is Database Testing?

- Database Testing is checking the **schema, tables, triggers, etc.** of the database under test.
- It may involve **creating complex queries** to **load/stress** test the database and check its responsiveness.
- It Checks data **integrity** and **consistency**.

What is JSON

- JavaScript Object Notation
- Lightweight data format akin to XML
- Self-describing
- Converts directly to JavaScript objects
- Easy to work with in JavaScript than other formats